

Practice: C4 Model

Think of the C4 model like Google Maps for software. We don't look at street views when we want to see a map of the continent. C4 gives us four distinct zoom levels: **Context (Level 1)**, **Container (Level 2)**, **Component (Level 3)**, and **Code (Level 4)**. For this practice, we will focus on Levels 1, 2, and 3, which you will use most in your engineering careers.

This practice walks you from **Beginner** to **Advanced with** complete step-by-step solutions and clean architectural blueprints using Mermaid syntax.

Practice 3: "EcoPulse" Smart Home Energy Monitor

The Scenario

EcoPulse is an industrial Internet of Things (IoT) software application. Hardware smart meters installed inside customer electrical panels emit real-time power metrics over MQTT to a centralized cloud system. Homeowners interact with a web platform to view power consumption metrics and predictive billing forecasts. The cloud backend leverages an external AI Analysis service to process usage patterns and generate automated energy-saving tips.

Beginner Level: System Context Diagram (Level 1)

Step-by-Step Task Instructions

1. Identify the human operator and the physical hardware devices interacting with the platform.
2. Treat EcoPulse as a closed system block.
3. Draw dependencies showing where data originates (the hardware) and where it gets processed by external specialized intelligence.

Solution & Diagram

- **Actors/Devices:** Homeowner, Physical Smart Meter (IoT Device).
- **External Systems:** External AI Analysis Engine.
- **Core System:** EcoPulse Energy Analytics Platform.

Code snippet

```
classDiagram

class homeowner{

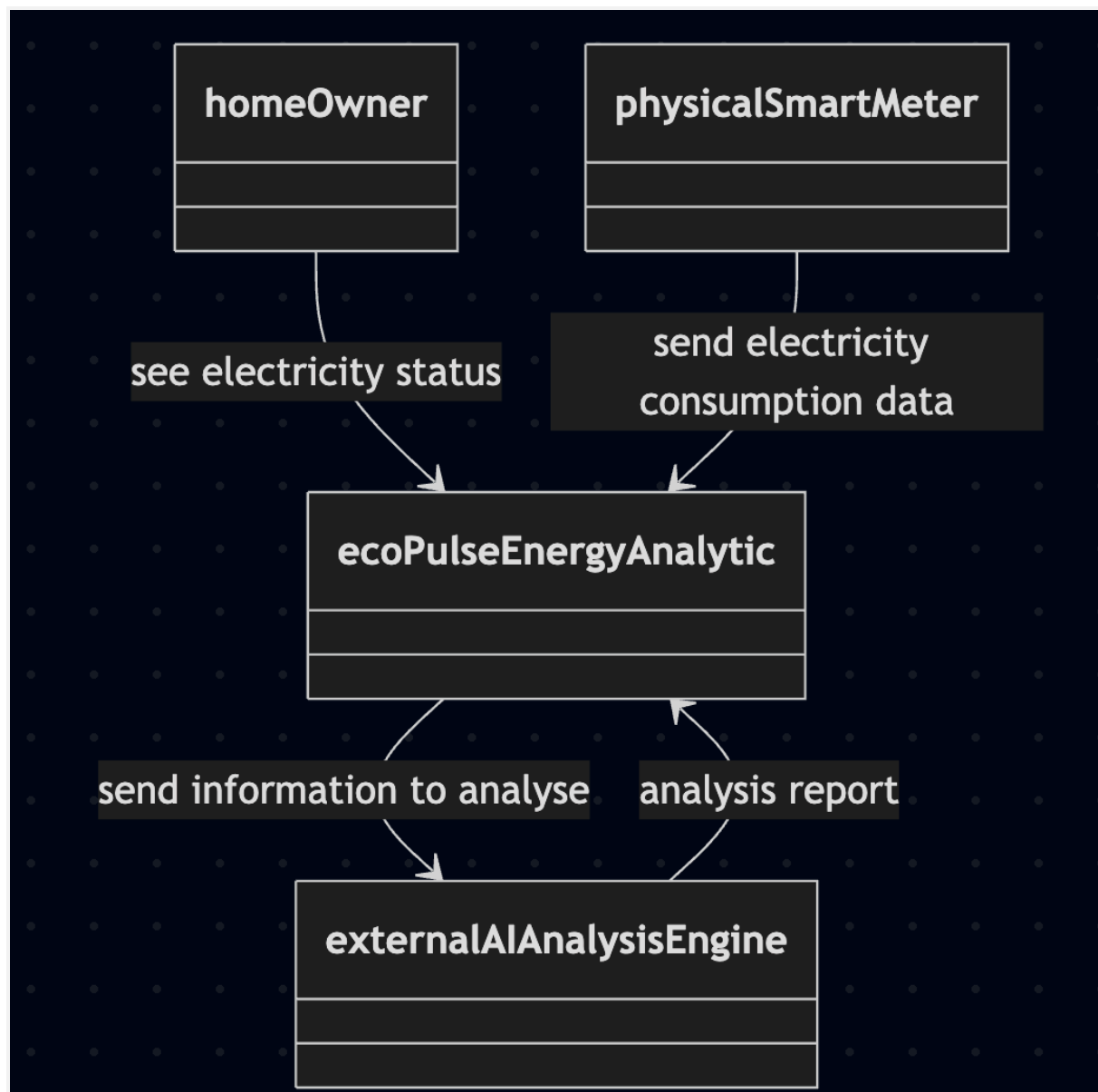
}
class physicalSmartMeter{

}
class externalAIAnalysisEngine{

}
class ecoPulseEnergyAnalytic{

}

homeowner --> ecoPulseEnergyAnalytic : see electricity status
physicalSmartMeter --> ecoPulseEnergyAnalytic : send electricity consumption data
ecoPulseEnergyAnalytic --> externalAIAnalysisEngine : send information to analyse
ecoPulseEnergyAnalytic <-- externalAIAnalysisEngine : analysis report
```



● Intermediate Level: Container Diagram (Level 2)

Step-by-Step Task Instructions

1. Differentiate how high-frequency time-series data is ingested versus how users read low-frequency dashboard configurations.
2. Incorporate a time-series database to track billions of sensor metrics, keeping relational metadata in a separate database.
3. Specify messaging protocols (e.g., MQTT vs HTTP).

Solution & Diagram

- **Containers:** Smart Meter Firmware (C++ Embedded), Cloud Ingestion Gateway (Java/Spring Boot), Web App Frontend (Next.js), Time-Series Database (InfluxDB), App Relational DB (MySQL).

Code snippet

```
classDiagram
class homeOwner{

}
class physicalSmartMeter{

}
class externalAIAnalysisEngine{

}
class smartMeterFirmware{

}
```

```
namespace ecoPulseEnergyAnalytics{
```

```
    class cloudIngestionGateway{
```

```
    }
```

```
    class webAppFrontEnd{
```

```
    }
```

```
    class timeSeriesDB{
```

```
    }
```

```
    class appRelationalDB{
```

```
    }
```

```
}
```

```
physicalSmartMeter --> smartMeterFirmware : uses
```

```
smartMeterFirmware --> cloudIngestionGateway : send raw data
```

```
cloudIngestionGateway --> timeSeriesDB : store electricity data
```

```
timeSeriesDB --> webAppFrontEnd : display data
```

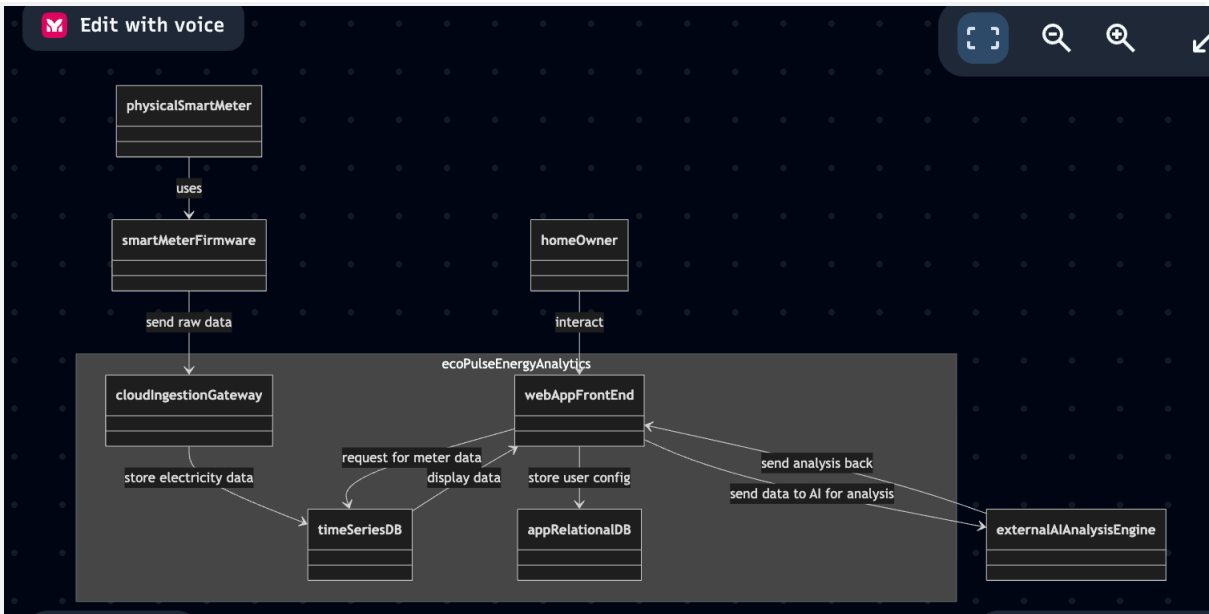
```
webAppFrontEnd --> timeSeriesDB : request for meter data
```

```
webAppFrontEnd --> appRelationalDB : store user config
```

```
homeOwner --> webAppFrontEnd : interact
```

```
webAppFrontEnd --> externalAIAnalysisEngine : send data to AI for analysis
```

```
externalAIAnalysisEngine --> webAppFrontEnd : send analysis back
```



● Advanced Level: Component Diagram (Level 3)

To round out our architectural overview of **EcoPulse**, let's examine the user-facing side of our smart home energy monitor.

When you design the frontend structure for a high-frequency Internet of Things (IoT) dashboard, the key challenge is **rendering stability**. If your UI components re-render every single time a tiny telemetry data packet arrives from an electrical meter, the user's web browser will freeze up. We use a Next.js framework because its architecture lets us separate server-side fetching from high-frequency, client-side visual component re-renders.

● Component Diagram: Cloud Ingestion Gateway (Java/Spring Boot)

Goal: Zoom into the **Cloud Ingestion Gateway (Java/Spring Boot)** to explore how high-throughput data packets are validated, processed, and routed under pressure.

Step-by-Step Task Instructions

1. Map internal components handling inbound networking threads.
2. Outline internal components that check message formats and filter noise before storage.

Solution & Diagram

- **Components:** MQTT Listener Broker, Stream Validator, Batch Writer, Device Authenticator Module.

Code snippet

flowchart TB

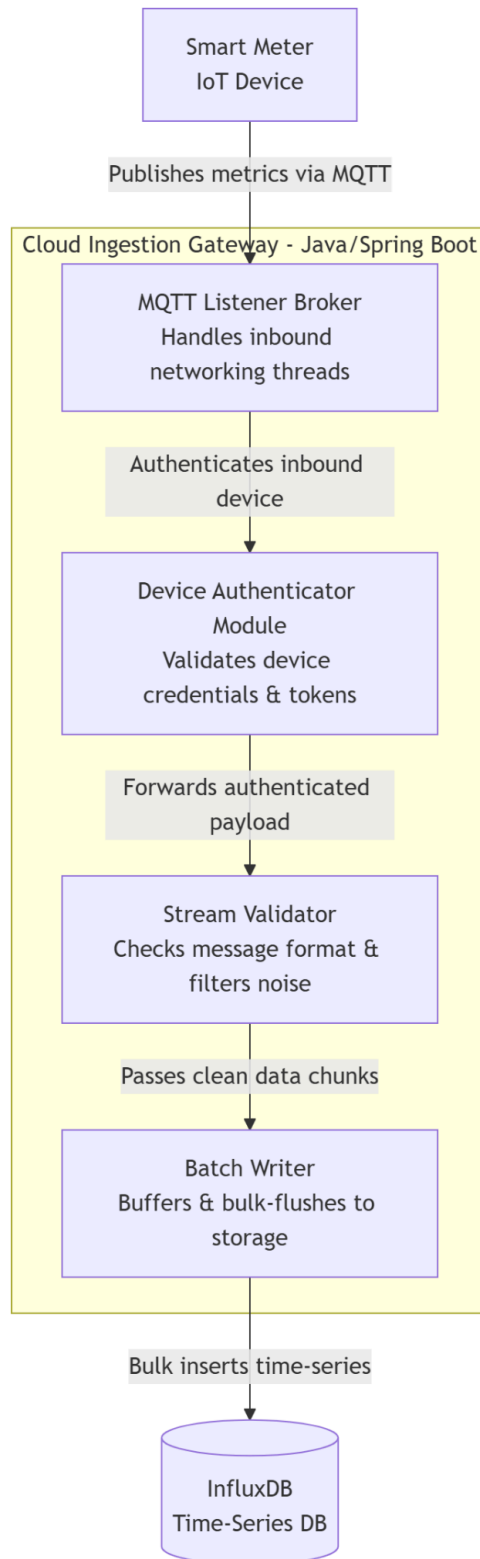
SmartMeter[Smart Meter\nIoT Device]

subgraph GW [Cloud Ingestion Gateway - Java/Spring Boot]

```
MQTT[MQTT Listener Broker\nHandles inbound networking threads]
Auth[Device Authenticator Module\nValidates device credentials &
tokens]
Validator[Stream Validator\nChecks message format & filters noise]
Writer[Batch Writer\nBuffers & bulk-flushes to storage]
end
```

```
TSDB[(InfluxDB\nTime-Series DB)]
```

```
SmartMeter -->|Publishes metrics via MQTT| MQTT
MQTT -->|Authenticates inbound device| Auth
Auth -->|Forwards authenticated payload| Validator
Validator -->|Passes clean data chunks| Writer
Writer -->|Bulk inserts time-series| TSDB
```

Component Diagram: Web Dashboard UI (Next.js)

Step-by-Step Task Instructions

1. **Isolate the Container:** Focus strictly on the boundaries of the Next.js application runtime. Account for both the client-side user interface view components and the server-side proxy actions.
2. **Handle Real-Time Rendering Demands:** Split your layout views into specialized, modular chunks. Create separate UI blocks for time-series charts (write-heavy updates) and billing settings forms (static data inputs).
3. **Establish Data Extraction Pipelines:** Map your React UI hooks to specialized API service clients. These clients act as dedicated proxies, querying the background InfluxDB database or uploading batch data metrics to the external AI Engine securely.

Solution Description & Diagram

- **UI Component Layer:** Renders the application canvas. Includes the **Real-Time Chart Widget** (renders power usage graphs), the **Predictive Savings Panel** (displays optimization suggestions), and the **Billing Settings Form** (manages target account thresholds).
- **State & Data Hook Layer:** Governs active application state logic. The `useTelemetry` hook manages active data arrays, while the `useAuth` context controls active user session states.
- **Server Actions & API Clients Layer:** The network transport framework. Contains the **Time-Series API Proxy** (handles high-velocity matrix lookups), the **Profile Async Client** (handles SQL data mutations), and the **AI Analysis Client Wrapper** (formats optimization request structures).

Code snippet

```
graph TD
    subgraph NextJS_Runtime ["Next.js Application Runtime"]

        subgraph UI_Layer ["UI Component Layer (Client-Side)"]
```

```

    Chart["Real-Time Chart Widget<br/>(Write-Heavy Charts)"]
    Savings["Predictive Savings Panel<br/>(Optimization View)"]
    Billing["Billing Settings Form<br/>(Static Data Input)"]
end

subgraph State_Layer ["State & Data Hook Layer"]
    Telemetry["useTelemetry Hook<br/>(Active Data Management)"]
    Auth["useAuth Context<br/>(Session Management)"]
end

subgraph Server_Layer ["Server Actions & API Clients (Proxy)"]
    TS_Proxy["Time-Series API Proxy<br/>(High-Velocity Lookups)"]
    Profile_Client["Profile Async Client<br/>(SQL Data Mutations)"]
    AI_Wrapper["AI Analysis Client Wrapper<br/>(Req Formatting)"]
end

subgraph Infrastructure ["External Infrastructure"]
    InfluxDB[(InfluxDB)]
    SQL_DB[(SQL Database)]
    AI_Engine[External AI Engine]
end

Chart --- Telemetry
Savings --- Telemetry
Billing --- Auth

Telemetry -->|Stream| TS_Proxy
Telemetry -->|Batch Metrics| AI_Wrapper
Auth -->|User Mutations| Profile_Client

TS_Proxy <--> InfluxDB
Profile_Client <--> SQL_DB
AI_Wrapper --> AI_Engine

```

